MICROCURRÍCULO: FUNDAMENTOS DE SOFTWARE
Folio: MC-2026-0001

1. DATOS DE IDENTIFICACIÓN
Asignatura: Fundamentos de Software
Código: ISW-201
Programa académico: Ingeniería de Software
Semestre / nivel: Tercer semestre
Créditos: 4
Horas: 32 teóricas, 32 prácticas, 64 de trabajo independiente (128 en total)
Prerrequisitos: Introducción a la Programación

2. JUSTIFICACIÓN
Fundamentos de Software introduce al estudiante de Ingeniería de Software en los conceptos, procesos y prácticas que estructuran la construcción de productos de software de calidad. La asignatura conecta la programación aprendida en cursos previos con la disciplina de ingeniería: el ciclo de vida, las metodologías ágiles y las pruebas. Aporta al perfil de egreso la capacidad de planificar, gestionar y verificar el desarrollo de software en equipos, con criterios de calidad y trazabilidad, base necesaria para asignaturas posteriores de arquitectura, gestión de proyectos y aseguramiento de la calidad.

3. OBJETIVOS O COMPETENCIAS
Generales:
- Aplicar los fundamentos del ciclo de vida del software, las metodologías ágiles y las pruebas para planificar y verificar el desarrollo de un producto de software en equipo.

Específicas:
- Distinguir las fases y modelos del ciclo de vida del software y seleccionar el más adecuado según el contexto del proyecto.
- Aplicar prácticas de metodologías ágiles, como iteraciones, historias de usuario y tableros de gestión, en la planificación de un proyecto.
- Diseñar y ejecutar pruebas de software, documentando casos y resultados con criterios de calidad verificables.

4. CONTENIDO TEMÁTICO
Unidad 1. Ciclo de vida del software
Resultados de aprendizaje:
- Distingue los modelos del ciclo de vida del software y justifica la elección de uno según las características del proyecto.

Temas:
- Modelos de ciclo de vida
  - Modelo en cascada y modelo incremental
  - Modelo en espiral y prototipado
- Fases y artefactos del proceso
  - Requisitos, análisis, diseño, implementación y mantenimiento
  - Documentación y trazabilidad de artefactos

Unidad 2. Metodologías ágiles
Resultados de aprendizaje:

- Aplica prácticas ágiles de planificación y seguimiento para organizar el trabajo de un equipo de desarrollo.

Temas:

- Marco ágil y Scrum
  - Valores y principios del manifiesto ágil
  - Roles, eventos y artefactos de Scrum
- Gestión del trabajo ágil
  - Historias de usuario y criterios de aceptación
  - Tableros de tareas e iteraciones

Unidad 3. Pruebas de software

Resultados de aprendizaje:

- Diseña casos de prueba y registra sus resultados para verificar la calidad de un componente de software.

Temas:

- Fundamentos de pruebas
  - Niveles de prueba: unitarias, de integración y de sistema
  - Pruebas de caja negra y caja blanca
- Diseño y documentación de pruebas
  - Casos de prueba y datos de entrada
  - Registro de defectos y trazabilidad de resultados

5. SECUENCIA DIDÁCTICA

Horas planificadas: 1.5 teóricas, 3 prácticas, 0 de trabajo independiente

Sesión 1 - Modelos de ciclo de vida (90 min, Teórica, Unidad 1)

INICIO

Docente: Presenta un caso de proyecto de software y pregunta qué etapas debería tener su desarrollo.

Estudiante: Responde y anota las etapas que propone para el caso.

Recursos: Diapositivas, Estudio de caso impreso

DESARROLLO

Docente: Expone los modelos en cascada, incremental, espiral y prototipado con ejemplos y compara sus ventajas.

Estudiante: Completa una tabla comparativa de los modelos con criterios de riesgo, alcance y entregas.

Recursos: Diapositivas, Plantilla de tabla comparativa

Docente: Organiza equipos y asigna a cada uno un escenario de proyecto para elegir un modelo.

Estudiante: Discute en equipo y justifica el modelo elegido para su escenario.

Recursos: Escenarios de proyecto, Jira para registrar la decisión en el backlog

CIERRE

Docente: Solicita que cada equipo exponga en un minuto el modelo elegido y su justificación.

Estudiante: Expone la elección y registra en Jira la decisión del equipo.

Recursos: Jira, Cronómetro

Sesión 2 - Marco ágil y Scrum (90 min, Práctica, Unidad 2)

INICIO

Docente: Pregunta cómo organizarían el trabajo de un equipo con entregas frecuentes y cambios de requisitos.

Estudiante: Comparte experiencias previas de trabajo en equipo y propone ideas.

Recursos: Pizarra, Tablero en Jira

DESARROLLO

Docente: Explica roles, eventos y artefactos de Scrum y modela la creación de un backlog de historias de usuario.

Estudiante: Redacta dos historias de usuario con criterios de aceptación para su proyecto.

Recursos: Jira, Plantilla de historia de usuario

Docente: Guía la creación del tablero y la planificación de una iteración de dos semanas.

Estudiante: Carga las historias en Jira y organiza el tablero con columnas de flujo.

Recursos: Jira, Guía de configuración de tablero

CIERRE

Docente: Revisa los tableros y retroalimenta la calidad de las historias.

Estudiante: Ajusta las historias según la retroalimentación y deja el tablero listo.

Recursos: Jira

Sesión 3 - Diseño y documentación de pruebas (90 min, Práctica, Unidad 3)

INICIO

Docente: Muestra un defecto real y pregunta cómo lo detectarían antes de liberar el software.

Estudiante: Propone formas de detección y anota ideas.

Recursos: Diapositivas, Ejemplo de defecto

DESARROLLO

Docente: Explica el diseño de casos de prueba con datos de entrada y resultados esperados.

Estudiante: Diseña tres casos de prueba para un componente de su proyecto.

Recursos: Plantilla de caso de prueba, Jira

Docente: Indica cómo registrar casos y defectos en el repositorio y el gestor de tareas.

Estudiante: Registra los casos en GitHub y los defectos como incidencias en Jira.

Recursos: GitHub, Jira

CIERRE

Docente: Verifica que los casos estén en GitHub y los defectos en Jira, y da retroalimentación.

Estudiante: Completa el registro y enlaza cada caso con la historia de usuario correspondiente.

Recursos: GitHub, Jira

6. METODOLOGÍA

- Estudio de caso: El docente presenta escenarios de proyectos de software reales o verosímiles; los equipos analizan el contexto, eligen modelos y prácticas, y justifican sus decisiones con criterios técnicos.
- Aprendizaje basado en proyectos: Los equipos desarrollan un proyecto integrador a lo

largo del curso: definen el ciclo de vida, planifican iteraciones en Jira y documentan pruebas en GitHub, con entregas parciales por corte.

7. SISTEMA DE EVALUACIÓN

Primer corte - 30%

- Informe de estudio de caso sobre ciclo de vida: Análisis de un escenario de proyecto con elección y justificación del modelo de ciclo de vida.
  Instrumento: Rúbrica analítica
  Herramientas: Jira
  Mide: Distingue los modelos del ciclo de vida y justifica la elección de uno según el proyecto.
- Taller de comparación de modelos: Tabla comparativa de modelos con criterios de riesgo, alcance y entregas.
  Instrumento: Lista de cotejo
  Mide: Distingue los modelos del ciclo de vida del software.

Segundo corte - 30%

- Backlog ágil en Jira: Historias de usuario con criterios de aceptación y planificación de una iteración en el tablero.
  Instrumento: Rúbrica analítica
  Herramientas: Jira
  Mide: Aplica prácticas ágiles de planificación y seguimiento para organizar el trabajo del equipo.
- Seguimiento de iteración: Registro del avance de la iteración con revisión del tablero y ajustes.
  Instrumento: Guía de observación
  Herramientas: Jira
  Mide: Aplica prácticas ágiles de planificación y seguimiento.

Tercer corte - 40%

- Proyecto integrador en GitHub: Entrega del proyecto con casos de prueba documentados y enlace a las incidencias en Jira.
  Instrumento: Rúbrica analítica
  Herramientas: GitHub, Jira
  Mide: Diseña casos de prueba y registra sus resultados para verificar la calidad de un componente de software.
- Defensa del proyecto integrador: Presentación del equipo con justificación de decisiones de proceso y pruebas.
  Instrumento: Escala de valoración
  Herramientas: GitHub, Jira
  Mide: Aplica prácticas ágiles y diseña casos de prueba en un proyecto real.

Total: 100%

8. BIBLIOGRAFÍA Y RECURSOS

Obligatorias:

- Sommerville, Ian. Ingeniería de software. Pearson Educación, 2016, 10.ª edición [SIN CONFIRMAR]

Complementarias:

- Pressman, Roger S. Ingeniería del software: un enfoque práctico. McGraw-Hill, 2015, 8.ª edición [SIN CONFIRMAR]

Recursos y herramientas:
- GitHub (usada en 3)
- Jira (usada en 8)
- Repositorios de documentación de proyectos de software (sin uso declarado)

NOTA: 2 referencia(s) marcadas [SIN CONFIRMAR]. Verificar año, edición e ISBN antes de presentar el documento.